

Mecha-Basketball 3D

Technical Report

Alessandro Carotenuto

MSc Artificial Intelligence & Robotics, Sapienza University of Rome

Interactive Graphics Course, Prof. Marco Schaerf

July 2026

Repository

<https://github.com/SapienzaInteractiveGraphicsCourse/final-project-intentionallyblankname>

Live demo

<https://sapienzainteractivegraphicscourse.github.io/final-project-intentionallyblankname/>

Contents

1	Project Overview	2
1.1	Robot Classes and Capabilities	2
2	Project Structure	3
3	Development Environment, Libraries and Assets	3
4	Architecture and Recurring Patterns	4
5	Hierarchical Models	5
6	Materials, Lighting and Textures	5
6.1	Lights	5
6.2	Post-Processing	6
6.3	Textures	6
6.4	Procedural Skybox	6
7	Animation	7
8	Physics, Collisions and Scoring	8
9	Audio	8
10	User Interaction and Controls	8
11	1v1 Mode and Enemy AI	9

1 Project Overview

Mecha-Basketball 3D is a Three.js web videogame about **Robots** and **Basketball**. The user could practice or play against the AI, it has three distinct robot classes which can use or play against.

Each class is built in Three.js from primitives, organized as hierarchical **THREE.Group** trees and animated procedurally. Two assets are external: the court (with most of its element) and the ball (both GLTF from **Sketchfab**).

1.1 Robot Classes and Capabilities

Class	Locomotion	Special Move	Spd	Sht	Stl	Blk
AMR Manipulator	Wheels	Dash	3	1	3	3
Legged Manipulator	Legs	Jump	2	3	2	5
Drone	Flight (rotors)	Flight (grab-and-rise)	5	2	1	1



Figure 1: Enter Caption

2 Project Structure

The codebase is organized by domain under `src/`. `main.js` is the only top-level file: it owns the scene, the render loop and every piece of global state, and wires the other modules together (Section 4).

```
src/
  main.js          scene, camera, lights, render loop, HUD wiring
  SharedEnums.js   frozen enums: GameMode, Team, TimeOfDay, ROBOT_KEYS
  robots/
    RobotBase.js    shared FSM, stats, locomotion/aim hooks
    AMRManipulator.js wheeled class, Dash
    LeggedManipulator.js legged class, Jump
    Drone.js         flying class, Flight
    ModelMakers/     procedural geometry factories (one per class)
  gameplay/
    Basketball.js    ball wrapper, HANDLED / FREE_SHOT / FREE
    BallPossession.js dribble state machine, handling, pickup
    ShootingSystem.js shot physics, hoop assist, scoring, preview
    CollisionWorld.js sphere-vs-AABB, sphere-vs-torus
    CombatMoves.js   steal, block
    EnemyAI.js        1v1 tactical state machine
  ui/MainMenu.js    menu navigation, gameplay reset
  debug/            tuning panel, collision wireframe overlays
  audio/SoundEffects.js synthesized Web Audio sound effects
  utils/            math helpers, physical constants
  public/models/     court + ball GLTF, the only external assets
```

Every class has a factory function that acts as a `ModelMaker` and then a class as **wrapper** which includes other properties like **stats**.

3 Development Environment, Libraries and Assets

Built with **Three.js** on **Vite**, in plain JavaScript. Three.js was chosen over raw WebGL to spend effort on modelling and animation rather than shader/buffer boilerplate. The renderer targets soft, physically plausible shadows: `PCFSoftShadowMap`, ACES filmic tone mapping, sRGB output.

Beyond the Three.js core, only official addons are used: `GLTFLoader`, `PointerLockControls`, the `EffectComposer` post-processing stack and the procedural Sky skydome.

Two external, pre-made assets:

- **Court** (Sketchfab, modified): a custom `GLTFLoader` plugin re-implements `KHR_materials_pbrSpecularGlossiness` (removed from Three.js r152+); some material colors were hand-edited in the raw `scene.gltf`
- **Basketball**: a dedicated GLTF with color, normal and metalness/roughness maps.

4 Architecture and Recurring Patterns

`main.js` owns all shared **mutable state** (`manipulator`, `basketball`, `cameraState`, ...) and is the only file allowed to import from every other module; every gameplay module receives what it needs through a single *context object* instead of importing from `main.js`, which would create circular dependencies:

Both the active robot and the basketball are passed as zero-argument *getter functions* (`getManipulator()`, `getBasketball()`) instead, so every consumer always reads the current value.

The render loop drives two independent dispatch chains, one per side (player, enemy). Some of the code patterns that are used:

- **Finite state machines:** robot state (`DRIBBLE/HANDLING/NO_BALL`), ball state (`HANDLED/FREE_SHOT/FREE`), the dribble (`push/drop/rise`), the shot (`windup/release/recover`) and combat moves (`reach/resolve`) are all plain objects with a `phase` string and a `phaseT` timer, advanced by hand each frame. Enums are frozen plain objects (`Object.freeze`), JavaScript has no native enum.
- **Fixed-timestep accumulator:** the dribble integrates at a constant 120 Hz and the shot flight at 240 Hz, decoupled from the variable render `delta`, via an accumulator.
- **Scratch objects:** every hot-path function (collision resolution, trajectory preview, ball spin) reuses `THREE.Vector3/Box3` instances allocated once, instead of allocating per call; the trajectory preview alone can call the collision resolver up to 2400× per frame while aiming.
- **Composition over inheritance for robots:** `RobotBase` is a class that wraps a plain factory function (`Object.assign(this, factory())`), keeping procedural geometry construction (never touched once stable) separate from gameplay state (stats, FSM, special moves).

5 Hierarchical Models

All three robot classes carry the same 3-revolute-joint arm above (or, for the Drone, hanging below) their locomotion base:

base (yaw) → link1 → elbow (pitch) → link2 (tapered) → wrist (pitch) → end effector + paddle

Each joint is structured as a nested `THREE.Group`, so rotating a parent carries every child with it. `link2` uses a tapered cross-section from a 4-sided `CylinderGeometry` turning that into a pyramid. The ball's resting point during handling is the convergence point of the two paddle plates' normals (otherwise not realistic) at distance $d/\cos(\alpha/2)$ from the wrist, not the paddle's flat surface center, which visibly interpenetrated the ball.

Per-class hierarchy below the arm:

- **Mobile Manipulator:** 4 wheels (tori) under a disc chassis; the wheel group's scale derives from the disc radius, so resizing the chassis keeps the wheels attached.
- **Legged Manipulator:** 4 two-joint legs (hip + knee) at the wheel offsets, each oriented radially outward; 25% larger overall. The flat foot pivots at its rear edge, the same off-center-pivot trick as the paddle.
- **Drone:** a body with four continuously spinning rotor arms (alternating spin direction between neighbors); the arm hangs upside down through a fixed flip group (`rotation.x =  $\pi$`) between the yaw base and the first link.

`RobotBase` wraps each procedural factory, owning the shared `RobotState` FSM, the stat-derived `speed` getter, and empty special-move hooks each subclass overrides.

6 Materials, Lighting and Textures

6.1 Lights

1 `HemisphereLight`, 1 `DirectionalLight` (4096² shadow map, frustum sized to cover the court), 4 `PointLight` (lamp posts) and 2 `SpotLight` (hoop fixtures, lit only at Sunset/Night). Colors, intensities and sun position follow a `TimeOfDay` preset table (Sunrise/Day/Sunset/Night).



Figure 2: Night Lights

6.2 Post-Processing

EffectComposer pipeline: `RenderPass` → `SSAOPass` → `OutputPass` → `SMAAPass`. `SSAOPass.minDistance/maxDistance` are normalized fractions of the near-far depth range, not world units; `camera.near` was raised from 0.1 to 5 against `far=5000` to stop depth precision from saturating; the kernel radius shrank from 90 to 12 to remove dark halos from surfaces reading as local occlusion behind a silhouette. `renderer.antialias` has no effect once rendering goes through the composer's targets, so `SMAAPass` runs last instead.

6.3 Textures

Two kinds: **downloaded** (ball color map, court diffuse map) and **procedurally generated** (ball normal/metalness/roughness; robot, bench and lamp-post normal/roughness/bump maps, built at runtime from a height field).

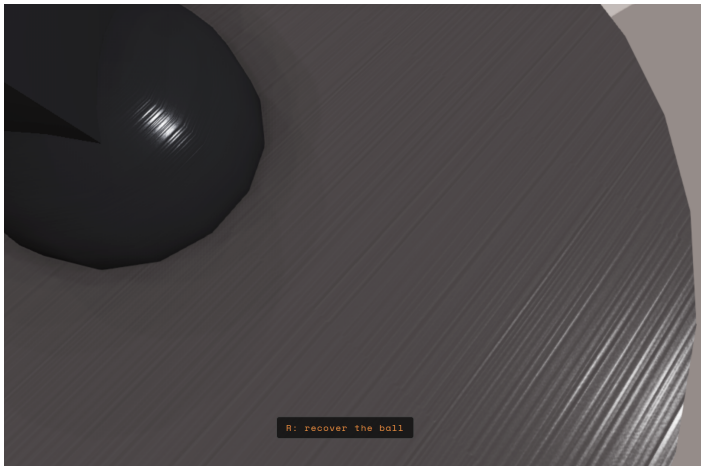


Figure 3: PBR Texture on AMR



Figure 4: Basketball Texture

6.4 Procedural Skybox

The flat background color was replaced with the `Sky` addon. The sun direction in the sky is always derived from the real `DirectionalLight` position, so the sky's glow and the cast shadows stay aligned by construction. At Night the sky mesh swaps to a flat dark color once the animated transition completes.

7 Animation

Every animation runs imperatively inside `requestAnimationFrame`; nothing is imported.

System	Technique
Automatic dribble	Push/drop/rise finite-state machine at fixed 120 Hz, synced with the elbow/link1 motion.
Shot flight	Projectile motion under constant gravity, sub-stepped at 240 Hz, full collision response.
Wheel steering	Exponential-smoothing angular interpolation, shortest path.
Legged gait	Diagonal-pair trot, advancing only while position actually changes frame to frame.
Drone flight feel	Body bank and thrust-tilt composed via quaternion multiplication, so the tilt axis stays attached to the nose at any yaw.
Special moves	Dash (6×-speed burst, 4 s cooldown), Jump (parabolic arc, 5 s cooldown), Flight (grab-rise-hold-descend, rigid ball attachment).
Shot animation	Windup/release/recover pose, elbow coupled to camera pitch while aiming.
Time-of-Day fades	2.5 s smoothstep cross-fade of every light/sky parameter.
Ball spin	Derived each frame from real displacement ($\text{axis} = \hat{u} \times \vec{v}$, $\text{rate} = \vec{v} /r$).

Third-Person Camera Tracking

While dribbling, an orbit camera follows the robot (mouse yaw/pitch, `lookAt` pinned, pitch clamped to $\sim 3\text{--}51^\circ$). While aiming, it switches to a free orientation, yaw/pitch drive `camera.quaternion` directly, decoupled from position, allowing a look over the robot toward the hoop.

Transitions always interpolate (position `lerp`, rotation `slerp`) toward a target recomputed every frame, since the two formulas are structurally different and switching without this layer snaps visibly.



Figure 5: Aiming animation (HANDLING STATE)

8 Physics, Collisions and Scoring

No physics engine: every collision test is a closed-form formula, **sphere vs. AABB** (ball vs. backboard/walls/poles/benches, robot vs. static geometry) and **sphere vs. torus** (ball vs. rim), both using $\vec{v}' = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n}$ with a per-material restitution e (backboard 0.15, rim 0.3, walls/poles 0.55, benches 0.5).

Shot flight is sub-stepped at 240 Hz because at full speed a single frame can move the ball farther than the backboard is thick. Post-bounce collision cooldowns are tracked **per object**: a single global cooldown after a rim bounce would suspend the backboard check at the exact moment the deflected ball reaches it.

A hoop-assist potential field pulls an airborne shot toward the rim center, strength scaled by the shooter's **shooting** stat. It generates unrealistic behaviour but it makes so robots can have different specialties

Scoring: 2 points if the shooter was inside the three-point arc at the moment of release, 3 otherwise, arc radius read from the court GLTF's accessor data. A basket is detected by interpolating the exact crossing point of the rim plane between two physics samples; testing only the sample already past the plane misses steep, close-range shots that genuinely went in.

9 Audio

Five sound effects, all synthesized at runtime with the Web Audio API, no audio file shipped: **score** (two ascending sine notes), **bounce** (low downward-sweeping tone plus a low-pass noise transient), **shoot** (band-pass noise sweeping upward), **steal** (same technique sweeping downward, shorter), **block** (low-pass noise hit over a descending tone). Each has a short linear attack before its exponential decay; an instant jump to peak gain produces an audible click.

10 User Interaction and Controls

The Main Menu (HTML/CSS overlaid on the canvas) walks through GAMEMODES → ROBOT selection (a second opponent-selection screen in 1v1) → TIME OF DAY + START. All three classes are instantiated (hidden) at page load, so switching class is a visibility toggle, never a reload. Selection cards run the same dribble state machine as the real game on a live offscreen renderer.

Input	Effect
P	Debug panel: per-component geometry tuning, live camera readout.
M	Toggle Spectate (free-fly) / Play (third-person).
Mouse (Play)	Orbits the camera; aims the arm.
WASD (Play)	Move relative to camera facing; wheels/legs steer toward it.
Left Shift	Special move for the active class.
Right Mouse	Held during HANDLING : pauses the dribble, frees the camera to aim.
Left Click	Shoot: crosshair raycast direction, constant speed, 60% inside the arc.
Walk Near Ball	Automatic pickup, no key.
Q / E	Steal / Block (1v1, no ball).
Esc	Pause menu.
1-8	Toggle collision/contact wireframe overlays.

Shot strength is constant rather than charge-based: varying force at aim time would change the previewed trajectory while lining it up. A live trajectory preview (a 3D tube rebuilt every frame

from the same physics as the real flight) shows exactly where the shot will go, turning green when it crosses the hoop.

The debug panel exposes every tunable parameter as live sliders, plus a Copy Config button serializing the current geometry; the shipped default dimensions were tuned this way. An Animation Preview section can force any pose for inspection from a free camera angle.

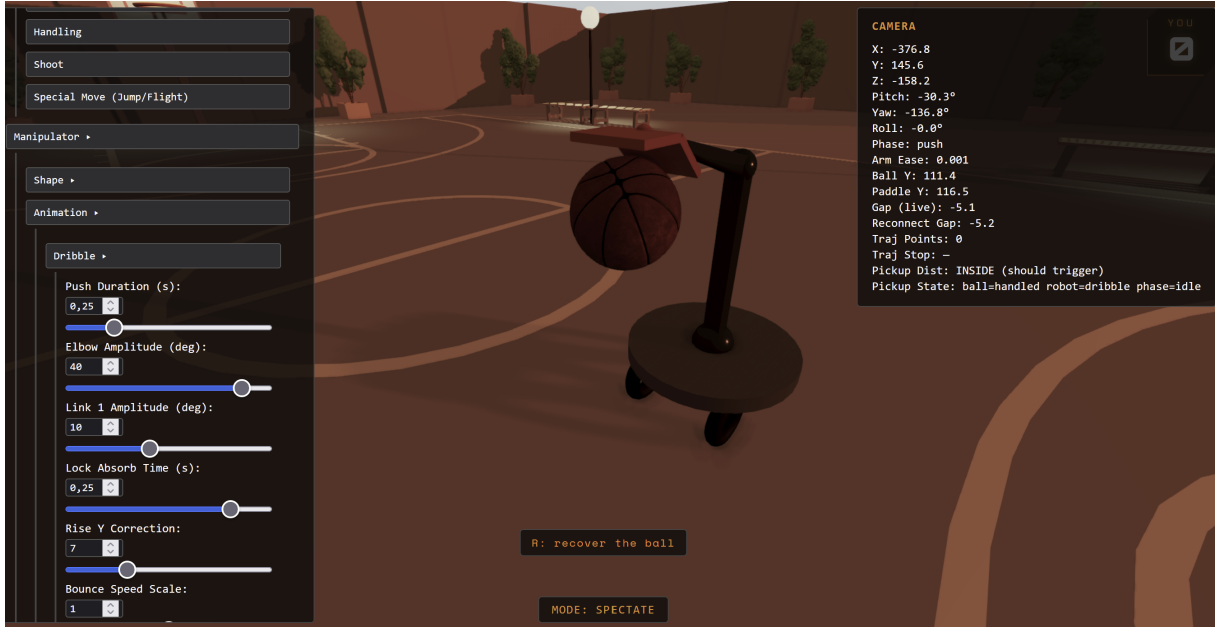


Figure 6: Debug Panel used for tuning

11 1v1 Mode and Enemy AI

Finally, in the 1v1 mode, the opponent runs a small tactical state machine, `EnemyState` $\in$ $\{\text{CHASE_BALL, ATTACK, DEFEND}\}$, recomputed every frame from ball possession: chase a loose ball, attack toward its hoop and shoot in range, or position between the player and the player's hoop while attempting Steal or Block. Shot elevation is solved with the ballistic-projectile equation at fixed speed, falling back to a 50° arc if the target is unreachable.

Steal and Block share one module between player and AI: a reach/sweep animation, a bounding-box contact test (asymmetric for Steal, wide margin in front of the sweeping robot, almost none behind), and a resolve step re-validating possession at the end of the sweep rather than at first contact. Robot-vs-robot overlap resolves with a possession-based asymmetric correction (the ball holder yields 25% of the separation, the other robot 75%).